

Architecture d'un service de production de vidéos assisté par l'IA

Emerson brings
sustainable
solutions to vital
industries

EMERSON

Emerson brings
sustainable
solutions to vital
industries

**Add your title
here. Make it
impactful**

**Add your title here.
Make it impactful**

Lumen5

- Plateforme lancée en 2017 par 3 co-fondateurs et amis à Vancouver
- Permet de convertir des articles de blogs en vidéo
- IA appliquée On applique des technologies plutôt que de développer nos propres modèles
- Financement «bootstrap»
- Le marché : Marketing B2B
- Génère environ 4000 vidéos par jour avec l'IA

Qui suis-je?

Antoine Morin-Paulhus

Développeur / Team lead dans l'équipe MARS.

MARS est une équipe full-stack qui s'occupe notamment de:

- La connection avec les partenaires de média stock.
- Plusieurs parties frontend de l'application.
- L'IA générative.
- Et plus encore!

Petit historique de l'AI dans la plateforme Lumen5

- **2017 - 2022**
 - NLP
 - Compréhension simple et résumés de phrases
- **2022**
 - ChatGPT apparaît
- **2023**
 - L'invasion des LLMs
 - Vidéos «talking head»
- **2024**
 - AI voiceover
- **2025**
 - Début de l'IA Générative dans la plateforme

L'AI appliquée en prod - Cas 1

Génération de script

Application des LLMs pour la génération et le raffinement de script de vidéo



< Back

Select a script

These video-ready scripts are generated from your content.

MOST SIMILAR

3 Reasons Why B2B Buyers Buy

🕒 Long (around 60 - 100 seconds)

Additional options

The Power of Emotional Marketing in B2B Content

🕒 Short (around 30 - 50 seconds)

The Better Way

🕒 Short (around 30 - 50 seconds)

[Show more](#)

AI Script Composer

TITLE	3 Reasons Why B2B Buyers Buy
INTRODUCTION	The buyer's journey has changed, and B2B buyers find the purchasing process difficult. But why do they make purchases? Let's find out.
SECTION: HOW HAS THE BUYER'S JOURNEY CHANGED?	Buying is more than just purchasing. It's a process with decision makers. But 75% of B2B buyers find it difficult or complex. Only 17% of their time is spent talking to humans. So, what can brands do?
SECTION: WHAT DO B2B BUYERS REALLY CARE ABOUT?	Buyers care about trust. They buy from those they trust. It's not about the product or price. They want to know, "How will this company support me?"
SECTION: 3 REASONS WHY B2B BUYERS BUY	1) Pain-based: Buyers have needs and want help fixing them.

Organisation des prompts dans le code

Nos prompts sont dans des fichiers .txt séparés, utilisant le langage de template django:

```
@inject_prompt_templates(
    [
        "script_generation/text_on_media_system.txt",
        "script_generation/voiceover_system.txt",
        "script_generation/user.txt",
    ]
)
def get_prompts_for_script_generation(
    # [...]
    text_on_media_system: str | None = None,
    voiceover_system: str | None = None,
    user: str | None = None,
    # [...]
) -> tuple[str, str]:
    """
    Returns the system and user prompts for generating a script!
    """
    # [...]
```

User Prompt

Voici un extrait de notre prompt user:

Subject: {{ subject|safe }}

Audience: {{ audience|safe }}

Author: {{ author|safe }}

Reference material: {{ reference_material|safe }}

Other instructions: {{ other_instructions|safe }}

System Prompt

Voici un extrait de notre prompt système:

```
You're a script writing assistant. Please write a script for a {{ script_type|safe }}  
social media video for the provided title (and description, if provided).
```

```
First outline the planned sections with target {{ unit|safe }} for each,  
adding to {{ target_description|safe }}. Then check if the {{ unit|safe }} count  
add up to the target - if not, generate the planned sections again.  
Then write the script.
```

```
Duration rules:
```

```
{% if script_type == "voiceover" %}
```

```
1. The script should be EXACTLY {{ target_words|safe }} words long.
```

```
[...]
```

C'est ici que notre magie opère! Nos prompts systèmes font en général 400-850 mots.

Evals

On utilise des LLMs pour évaluer la performance de nos prompts par rapport à plusieurs exemples.

Le concept:

1. On crée un ensemble de cas de test avec des exemples variés
2. Un LLM évalue les résultats selon plusieurs critères (échelle 1-5):
 - **Style Transfer Accuracy** - Est-ce que le style correspond?
 - **Content Preservation** - Est-ce que le contenu est préservé?
 - **Content Grounding** - Y a-t-il des hallucinations?
3. On agrège les scores pour mesurer la performance globale

Cela nous permet d'itérer sur nos prompts de manière mesurable.

L'AI appliquée en prod - Cas 2

Recherche vectorielle

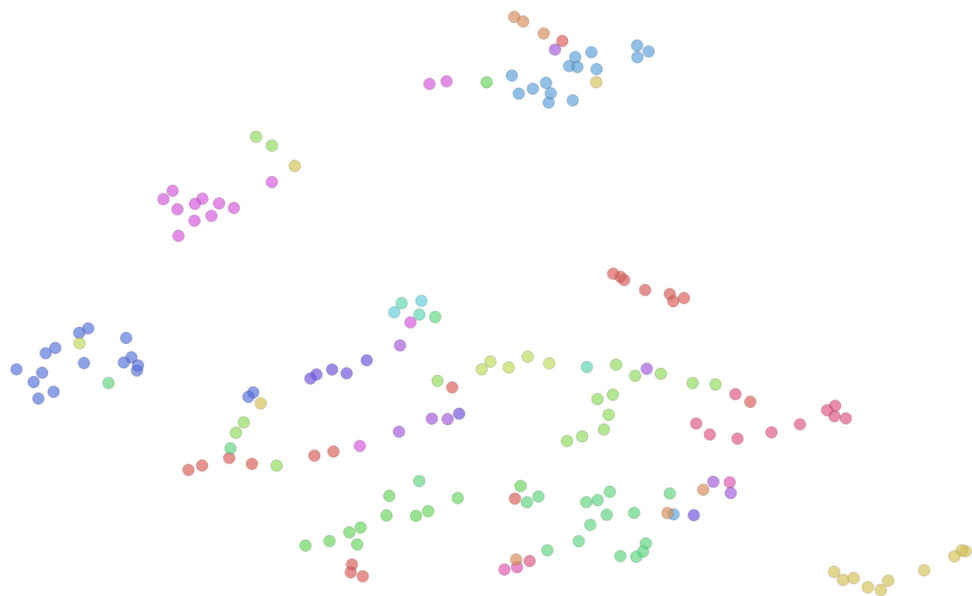
Recherche vectorielle avec QDrant

Exemple: Trouver des images similaires en utilisant des embeddings d'image

Embedding

<https://platform.openai.com/docs/guides/embeddings>

Cluster de vecteurs similaires



Recherche d'image similaires

Comment ça fonctionne

- On trouve ou crée des représentation textuelles d'image.
- On utilise l'API d'embedding d'OpenAI pour créer des vecteurs qui représentent chaque image de nos collections.
- On cherche les images dont les vecteurs sont les plus près dans notre DB Qdrant

Pour la recherche par texte:

- On convertit la requête en embedding et encore une fois, on cherche les vecteurs les plus près du vecteur de la requête

Exemple live de qdrant:

- <https://food-discovery.qdrant.tech/>

Recherche d'image similaires

Requête d'embedding via OpenAI

```
import OpenAI from "openai";
const openai = new OpenAI();

const embedding = await openai.embeddings.create({
  model: "text-embedding-3-small",
  input: "Your text string goes here",
  encoding_format: "float",
});

console.log(embedding);
from openai import OpenAI
client = OpenAI()

response = client.embeddings.create(
  input="Your text string goes here",
  model="text-embedding-3-small"
)

print(response.data[0].embedding)
```

Exemple tiré de <https://platform.openai.com/docs/guides/embeddings?lang=python>

Recherche d'image similaires

Résultat d'embedding via OpenAI

```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "index": 0,
      "embedding": [
        -0.006929283495992422,
        -0.005336422007530928,
        -4.547132266452536e-05,
        -0.024047505110502243
      ]
    }
  ],
  "model": "text-embedding-3-small",
  "usage": {
    "prompt_tokens": 5,
    "total_tokens": 5
  }
}
```

Recherche d'image similaires

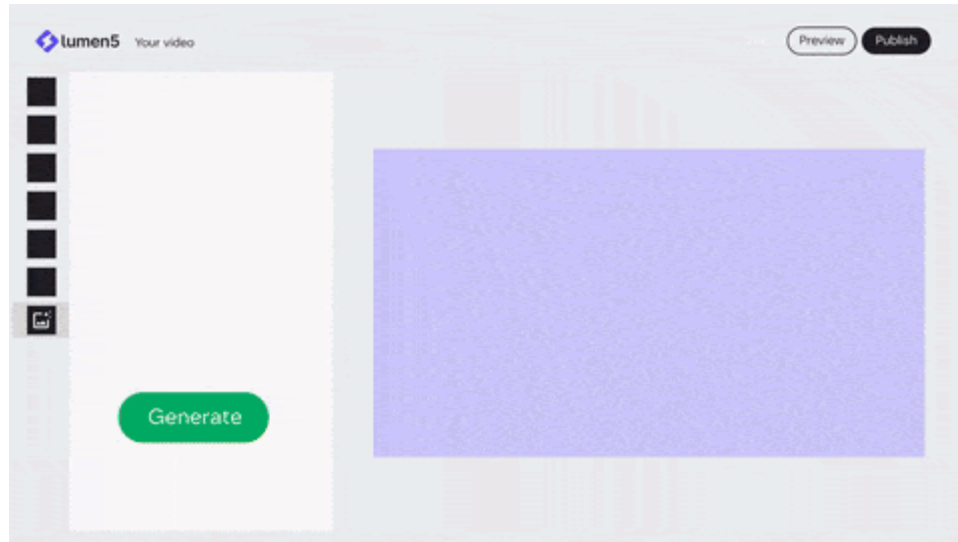
Pistes d'amélioration

- Utiliser un vecteur d'embedding utilisé directement à partir d'une image.
- Non supporté par OpenAI pour le moment.
- Possibilité d'utiliser un modèle de vision tel que ConvNeXt

L'AI appliquée en prod - Cas 3

IA Générative

Les clients exigent de plus en plus d'avoir accès à des outils d'IA générative.



L'AI appliquée en prod - Cas 3

IA Générative

Démo / Exercise

LIVE DEMO FAL.AI

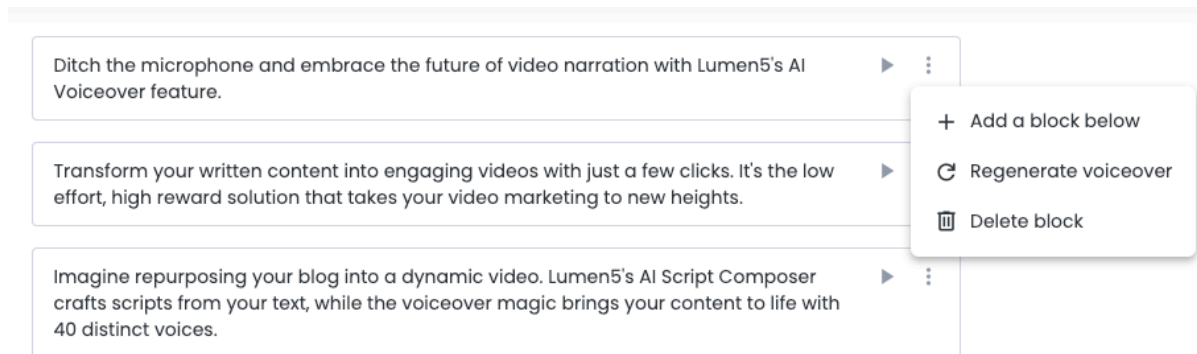
- Génération d'une image
- Modèles Image -> video
- Génération de modèles 3D

L'AI appliquée en prod - Cas 4

Des voix artificielles

Voiceover avec Elevenlabs

Dans lumen5:



- Elevenlabs:
 - **Input:** du texte écrit
 - **Output:** un fichier audio avec une voix artificielle

L'AI appliquée en prod - Cas 4

Des voix artificielles

Les voix artificielles, opportunités et risques:

- Vidéos lues par l'IA (Lumen5 et autres)
- Fraude par clone de la voix
- Donner la voix à ceux qui n'en ont pas ou plus

À écouter:

Ces secrets que nos voix livrent à l'IA 🎧

Mini challenge

Comment assurer un maximum de concurrence sans dépasser le rate-limit de ElevenLabs?

Problème:

Les fournisseurs tels qu'ElevenLabs permettent seulement X requête par minute.

Comment maximiser notre efficacité sans dépasser les limites?

Comment résoudre?

Réponse à la prochaine page 👁👁

Mini challenge

Comment assurer un maximum de concurrence sans dépasser le rate-limit de ElevenLabs?

Exemple de solution:

- Utiliser un sémaphore `asyncio.Semaphore`

```
import asyncio

async def worker(semaphore, id):
    async with semaphore:
        print(f"Worker {id} acquired semaphore")
        await asyncio.sleep(1) # Simulate some work
        print(f"Worker {id} released semaphore")

async def main():
    semaphore = asyncio.Semaphore(2) # Allow 2 concurrent workers
    tasks = [worker(semaphore, i) for i in range(5)]
    await asyncio.gather(*tasks)

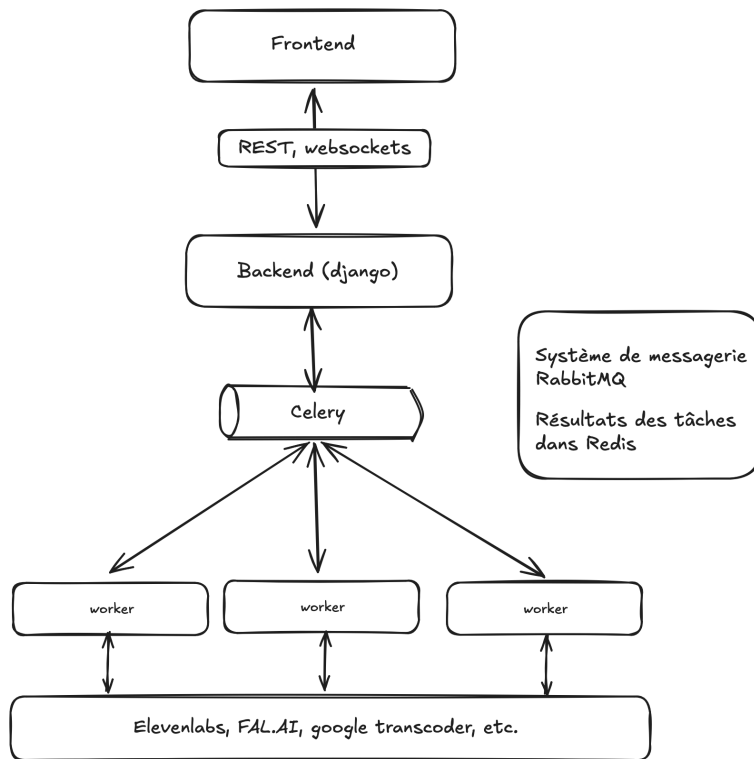
if __name__ == "__main__":
    asyncio.run(main())
```

Output

```
python3.11 semaphores.py  
Worker 0 acquired semaphore  
Worker 1 acquired semaphore  
Worker 0 released semaphore  
Worker 1 released semaphore  
Worker 2 acquired semaphore  
Worker 3 acquired semaphore  
Worker 2 released semaphore  
Worker 3 released semaphore  
Worker 4 acquired semaphore  
Worker 4 released semaphore
```

Architecture des services

Comment tout ceci est organisé?



Hébergement

Gestion du cluster

- GCP (Google Cloud Platform)
- On adopte une approche *Infrastructure as Code*
 - Terraform
 - Flux

Monitoring

- Prometheus
- AlertManager
- Sentry



Sécurité et utilisation responsable de l'IA

Comment encadrer l'utilisation de l'IA?

Les outils

- API de modérations des fournisseurs:
 - FAL.AI rend très difficile la génération de contenu NSFW
 - OpenAI donne accès à un API de modération.
- Fonctionnalités de vérification
 - Groundhog
 - Un outil supplémentaire pour valider l'information véhiculée dans nos scripts.

Le choix du marché

- Les communicateurs B2B utilisant notre plateforme créent du contenu en général plus professionnel.

Exemple de modération OpenAI

Exemple de classification de texte

```
from openai import OpenAI
client = OpenAI()

response = client.moderations.create(
    model="omni-moderation-latest",
    input="...text to classify goes here...",
)

print(response)
```

<https://platform.openai.com/docs/guides/moderation>

Exemple de modération FAL.ai

FAL.AI fait sa propre modération

1

Enable Safety Checker ⓘ

⚠

 The safety checker cannot be disabled on the playground. This property is only available through the API.

Output Format ⓘ

jpeg

▼

L'IA pour le développement

Nos outils

L'équipe a accès aux outils dev assistés par l'IA de son choix:

- Claude
- Cursor
- Copilot

Autres outils

- Revue de code par un agent Claude
- Création de PR par un agent Claude à partir de Jira

* Welcome to Claude Code

```
> Think harder...
while(curious) {
    question everything();
    dig_deeper();
    connect_dots(unexpected);

    if (stuck) {
        keep_thinking();
    }
}
```

Questions?

Merci!